

School of Engineering & Technology	
Department: SOET	Session: 2026-27
Program: BCA (AI&DS)	Semester: V
Course Code: ENCA351	Course Name: Design and Analysis of Algorithms Lab
Faculty: Dr. Aarti	Assignment: Capstone Assignment 1

Smart City Traffic Management and Route Optimization System

Capstone Assignment: Final Project Report

A graph-algorithm based system for route planning, traffic analysis, connectivity checks and infrastructure planning in a city road network

Submitted by	Shreyansh Vaibhaw
Roll number	2401201094
Program	BCA (AI&DS), Semester V
Submitted to	Dr. Aarti
GitHub repository	https://github.com/ShreyanshVaibhaw/smart-city-traffic-system-shreyansh-vaibhaw
Date	September 2026

Table of contents

Abstract	3
1. Introduction.....	3
1.1 Smart city concepts	3
1.2 Importance of traffic optimization	3
1.3 Problem statement and objectives	3
1.4 Tools and technologies.....	4
2. System design.....	4
2.1 The road network as a graph.....	4
2.2 Modules.....	4
2.3 The web application	5
2.4 Algorithms and where they are used	6
2.5 Implementation notes.....	6
3. Experimental results.....	7
3.1 Sample road network	7
3.2 Route planning with Dijkstra's algorithm	8
3.3 Traffic analysis with the Bellman-Ford algorithm	10
3.4 Network connectivity with BFS and DFS	13
3.5 Infrastructure planning with Prim's and Kruskal's algorithms	16
3.6 Measurements on networks of different sizes.....	18
4. Analysis.....	21
4.1 Comparison of graph algorithms.....	21
4.2 Traffic optimization observations.....	22
4.3 Infrastructure planning insights	22
4.4 Suitability of each algorithm for traffic management.....	22
5. Conclusion	23
5.1 Key findings	23
5.2 Benefits of graph algorithms in transportation systems.....	23
5.3 Future improvements.....	23
References.....	23
Appendix A: Repository contents.....	24
Appendix B: Running the project	24

Abstract

Traffic congestion is one of the most visible problems of fast-growing cities. This project models a city's road network as a weighted graph and uses classic graph algorithms to answer the questions that commuters and city administrators ask every day: what is the fastest route between two places, how does that route change when a road is congested or closed, which parts of the city can be reached from a given point, and which roads must be built to connect every intersection at the lowest cost. Dijkstra's algorithm, the Bellman-Ford algorithm, breadth first search, depth first search, Prim's algorithm and Kruskal's algorithm were implemented by hand in Python and wrapped in an interactive Streamlit application. A Jupyter notebook runs all algorithms on random cities with 10, 25, 50 and 100 intersections and compares execution time, memory and results. Dijkstra was two to three times faster than Bellman-Ford at every size, Prim was faster than Kruskal at every size, and BFS and DFS were indistinguishable, while every algorithm finished in well under a millisecond on the largest city.

1. Introduction

1.1 Smart city concepts

A smart city uses data and software to run its services better: water, power, waste, public safety and, most visibly, transport. Sensors on roads and in vehicles report where traffic is, control rooms watch the whole network at once, and navigation apps guide millions of individual trips. Behind all of these services sits the same abstraction: the road network is a graph. Intersections are vertices, roads are edges, and the time or distance needed to drive along a road is the weight of its edge. Once the city is represented this way, decades of well-understood graph algorithms become available to answer practical questions quickly and reliably.

This project builds exactly that representation and puts the most important algorithms behind an interactive interface, so that a commuter can ask for a route, an administrator can simulate congestion, and a planner can see which roads matter most.

1.2 Importance of traffic optimization

Congestion costs a city in several ways at once. Commuters lose time, vehicles burn fuel while idling, emissions rise, and emergency vehicles arrive later. Roads and junctions are expensive to build, so the existing network has to be used as well as possible. Traffic optimization tackles this from four directions, and each one maps onto a graph problem:

- **Route planning.** Find the least-cost route between two points (single-source shortest path).
- **Dynamic rerouting.** Recompute routes when road costs change because of congestion, accidents or closures, and quantify the extra cost (shortest path on an edited graph).
- **Reachability.** Know which intersections can still be reached from a point, and how far a disruption spreads (graph traversal).
- **Infrastructure planning.** Connect every intersection with the smallest total construction cost (minimum spanning tree).

1.3 Problem statement and objectives

The assignment asks for a Smart City Traffic Management System that models a city as a graph and uses graph algorithms to help commuters and administrators make efficient routing decisions. The objectives of this project are:

1. Represent the city as a weighted graph and display both the adjacency list and the adjacency matrix.

2. Compute the shortest route between a source and a destination with Dijkstra's algorithm and show every intermediate distance update.
3. Simulate congestion by changing road costs, recompute routes with the Bellman-Ford algorithm and compare the original and updated routes.
4. Explore the network from a starting intersection with breadth first search and depth first search and report the traversal order and the reachable intersections.
5. Generate the minimum spanning tree with Prim's and Kruskal's algorithms and report the selected roads and the total infrastructure cost.
6. Compare the algorithms experimentally on networks of 10, 25, 50 and 100 intersections.
7. Provide an interactive web application, a Jupyter notebook and this report.

1.4 Tools and technologies

Table 1: Tools and technologies used

Tool	Purpose in this project
Python 3.12	Implementation language for every algorithm and the application
Streamlit	Interactive web application with one page per module
NetworkX	Drawing the road network (the algorithms themselves are written by hand)
Matplotlib	Maps, route highlights and the charts of the experiments
pandas and NumPy	Adjacency matrix, result tables and the experiment data
Jupyter	Project notebook with the walk-through and the comparative analysis
Git and GitHub	Version control and submission of the private repository

2. System design

2.1 The road network as a graph

The city is stored in a class called RoadNetwork. Every intersection is a vertex, every road is an undirected edge (traffic flows both ways) and the travel cost of the road is the edge weight. Costs must be positive numbers; they can be kilometres or minutes, the algorithms do not care. Internally the graph is a dictionary of dictionaries, which is the classic adjacency list: roads["Central Square"] is itself a dictionary that maps each neighbouring intersection to the cost of the road between them.

The adjacency list needs space proportional to the number of intersections plus roads, and listing the neighbours of an intersection takes time proportional to its degree, which is exactly what the traversal and shortest path algorithms need. The adjacency matrix is produced on demand as a pandas DataFrame: it needs space proportional to the square of the number of intersections, but it answers "is there a road between A and B?" in constant time, and it is the representation most people find easiest to read. The application displays both, as the assignment requires.

2.2 Modules

Table 2: Modules of the system

Module	File	Algorithms	Purpose
Road Network Modeling	road_network.py	Adjacency list and matrix	Create or load a city, add intersections and roads, assign costs, save and load CSV files
Route Planning	route_planner.py	Dijkstra	Shortest route, total travel cost, every intermediate distance update
Traffic Analysis	traffic_analysis.py	Bellman-Ford	Congestion and closure simulation, route recomputation, before and after comparison
Network Connectivity	connectivity_module.py	BFS, DFS	Traversal order, reachable and unreachable

			intersections, regions of the city
Infrastructure Planning	mst_planner.py	Prim, Kruskal	Minimum spanning tree, selected roads, total infrastructure cost
Comparative Analysis	experiments.py	All of the above	Execution time and memory on cities of 10, 25, 50 and 100 intersections
Visualisation	visualizer.py	-	Map drawing and table formatting shared by the app, the notebook and this report
Web application	app.py	-	Streamlit interface that ties the modules together

2.3 The web application

The application is a single Streamlit script with a sidebar that selects one of six pages. The current city lives in the Streamlit session state, so a city built on the first page is available on every other page. The sample city is loaded automatically, which means every module shows a result the moment it is opened; the user can then change the source and destination, edit road costs or load a different city. Every result is recomputed live when an input changes, so there are no stale outputs. Figure 1 shows the Road Network page.

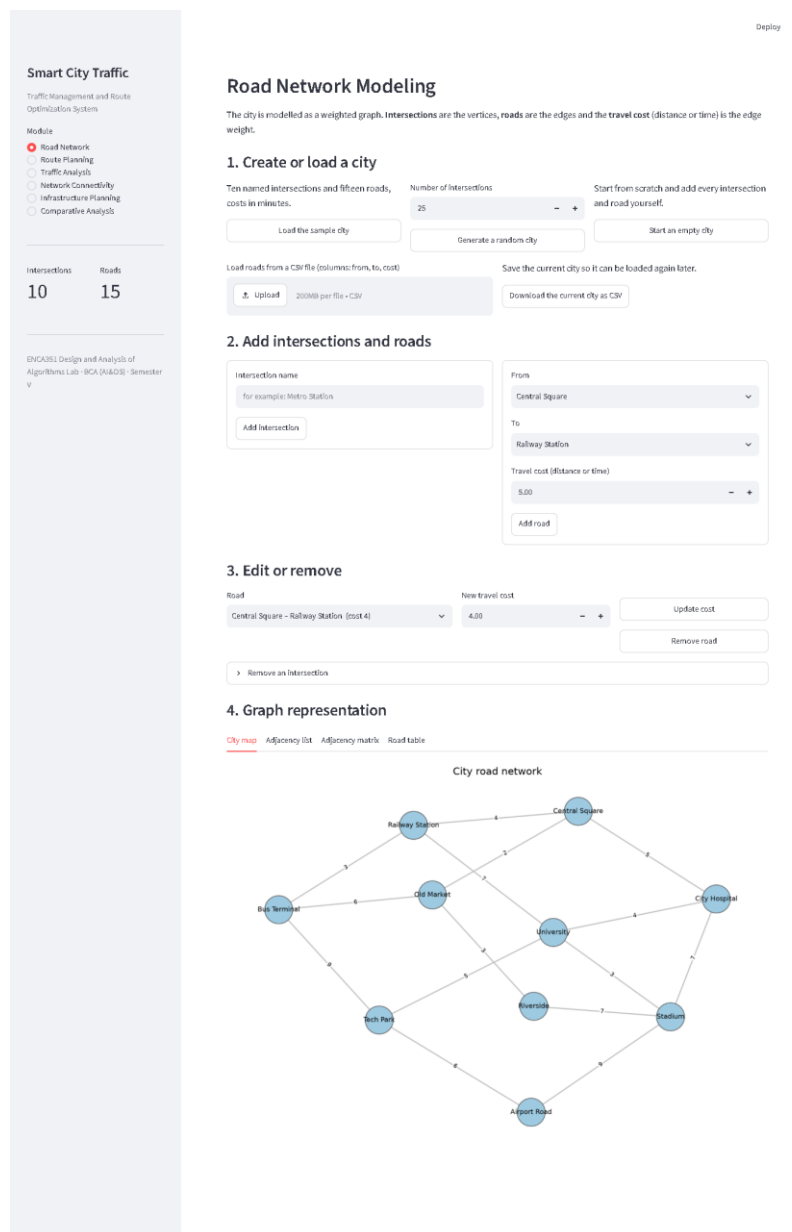


Figure 1: Road Network Modeling page of the application

2.4 Algorithms and where they are used

Table 3: Algorithms used in the system (V = intersections, E = roads)

Algorithm	Used in	Time complexity	Real-world application
Dijkstra	Route Planning	$O((V + E) \log V)$	Navigation systems, ride sharing, emergency routing
Bellman-Ford	Traffic Analysis	$O(V \times E)$, early stop	Dynamic traffic management, accident rerouting, closures
Breadth First Search	Network Connectivity	$O(V + E)$	Traffic monitoring, spread of a disruption
Depth First Search	Network Connectivity	$O(V + E)$	Network exploration, accessibility analysis
Prim	Infrastructure Planning	$O(E \log V)$	Road expansion, utility distribution
Kruskal	Infrastructure Planning	$O(E \log E)$	Smart city infrastructure development

2.5 Implementation notes

- **Dijkstra.** A binary heap (Python's `heapq`) holds (distance, intersection) pairs. Every time a shorter distance is found the update is recorded, and after every visited intersection a snapshot of the whole distance table is stored, so the application can show the working step by step.
- **Bellman-Ford.** Every road is relaxed in both directions in each pass. The loop stops as soon as a pass changes nothing, and one extra check would detect a negative cycle (it never fires here, because every road cost is positive). Congestion is applied to a copy of the city, so the original and the congested routes can be compared side by side.
- **BFS and DFS.** BFS uses a deque as its queue and records the queue after every step. DFS is the recursive version and records the current route from the start, which is the recursion stack.
- **Prim and Kruskal.** Prim keeps the candidate roads in a heap and adds the cheapest road that reaches a new intersection. Kruskal sorts all roads and uses a union-find structure with path halving to reject roads that would create a cycle. Both report every decision they make.
- **Measurements.** The experiments call every algorithm with the step recording switched off, so the timings measure the algorithm alone and not the bookkeeping used for display.

3. Experimental results

3.1 Sample road network

The sample city used throughout the application and the notebook has 10 intersections and 15 roads. The costs are travel times in minutes. The roads are listed in Table 4 and drawn in Figure 2.

Table 4: Roads of the sample city

From	To	Cost (min)
Central Square	Railway Station	4
Central Square	Old Market	2
Central Square	City Hospital	5
Railway Station	Bus Terminal	3
Railway Station	University	7
Old Market	Bus Terminal	6
Old Market	Riverside	3
City Hospital	University	4
City Hospital	Stadium	7
Bus Terminal	Tech Park	9
University	Tech Park	5
University	Stadium	3
Riverside	Stadium	7
Stadium	Airport Road	9
Tech Park	Airport Road	6

Sample city road network (costs in minutes)

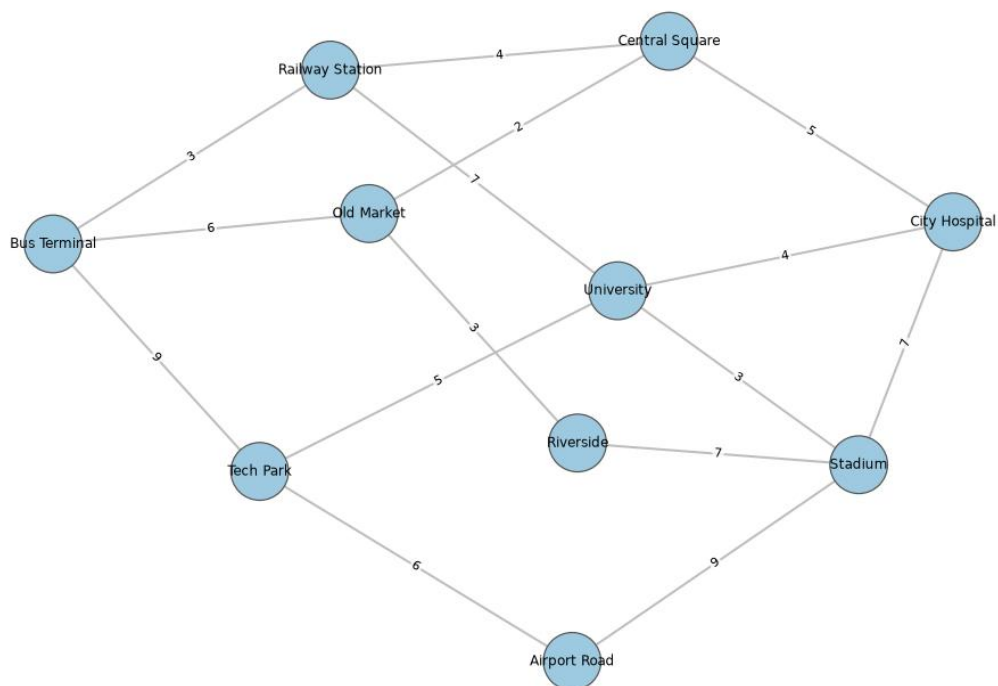


Figure 2: Sample city road network with travel costs in minutes

Table 5: Adjacency list of the sample city

Intersection	Neighbours (cost)
Central Square	City Hospital (5), Old Market (2), Railway Station (4)
Railway Station	Bus Terminal (3), Central Square (4), University (7)
Old Market	Bus Terminal (6), Central Square (2), Riverside (3)

City Hospital	Central Square (5), Stadium (7), University (4)
Bus Terminal	Old Market (6), Railway Station (3), Tech Park (9)
University	City Hospital (4), Railway Station (7), Stadium (3), Tech Park (5)
Riverside	Old Market (3), Stadium (7)
Stadium	Airport Road (9), City Hospital (7), Riverside (7), University (3)
Tech Park	Airport Road (6), Bus Terminal (9), University (5)
Airport Road	Stadium (9), Tech Park (6)

Table 6: Adjacency matrix of the sample city (0 = no direct road)

	Central Sq.	Railway St.	Old Market	Hospital	Bus Term.	University	Riverside	Stadium	Tech Park	Airport Rd.
Central Sq.	0	4	2	5	0	0	0	0	0	0
Railway St.	4	0	0	0	3	7	0	0	0	0
Old Market	2	0	0	0	6	0	3	0	0	0
Hospital	5	0	0	0	0	4	0	7	0	0
Bus Term.	0	3	6	0	0	0	0	0	9	0
University	0	7	0	4	0	0	0	3	5	0
Riverside	0	0	3	0	0	0	0	7	0	0
Stadium	0	0	0	7	0	3	7	0	0	9
Tech Park	0	0	0	0	9	5	0	0	0	6
Airport Rd.	0	0	0	0	0	0	0	9	6	0

3.2 Route planning with Dijkstra's algorithm

A commuter travels from Central Square to Airport Road. Dijkstra's algorithm finds the route Central Square → City Hospital → University → Tech Park → Airport Road with a total travel cost of 20 minutes, which is confirmed by adding the four road costs (5 + 4 + 5 + 6). The algorithm made 13 distance updates while settling all 10 intersections. Table 7 lists every update in the order it happened and Table 8 gives the final shortest cost from Central Square to every intersection.

Table 7: Intermediate distance updates made by Dijkstra's algorithm

Step	Visiting	Road to	Road cost	Old distance	New distance
1	Central Square	Railway Station	4	∞	4
2	Central Square	Old Market	2	∞	2
3	Central Square	City Hospital	5	∞	5
4	Old Market	Bus Terminal	6	∞	8
5	Old Market	Riverside	3	∞	5
6	Railway Station	Bus Terminal	3	8	7
7	Railway Station	University	7	∞	11
8	City Hospital	University	4	11	9
9	City Hospital	Stadium	7	∞	12
10	Bus Terminal	Tech Park	9	∞	16
11	University	Tech Park	5	16	14
12	Stadium	Airport Road	9	∞	21
13	Tech Park	Airport Road	6	21	20

Table 8: Shortest cost to every intersection

Intersection	Shortest cost from Central Square	Reached via
Central Square	0	-
Railway Station	4	Central Square
Old Market	2	Central Square
City Hospital	5	Central Square
Bus Terminal	7	Railway Station
University	9	City Hospital
Riverside	5	Old Market

Stadium	12	City Hospital
Tech Park	14	University
Airport Road	20	Tech Park

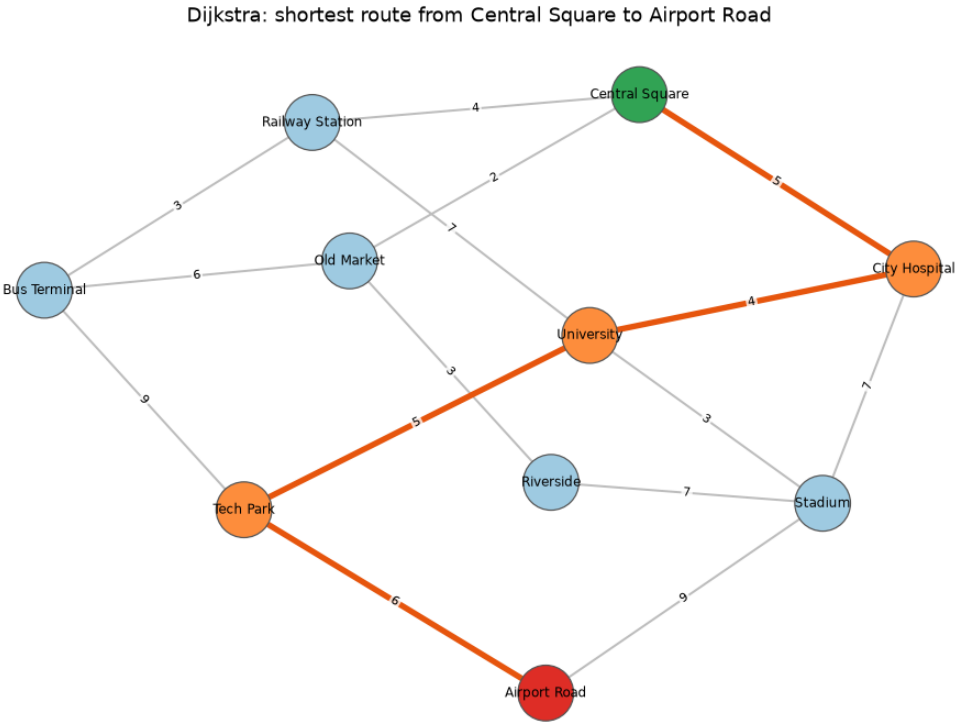


Figure 3: Shortest route from Central Square to Airport Road (orange)

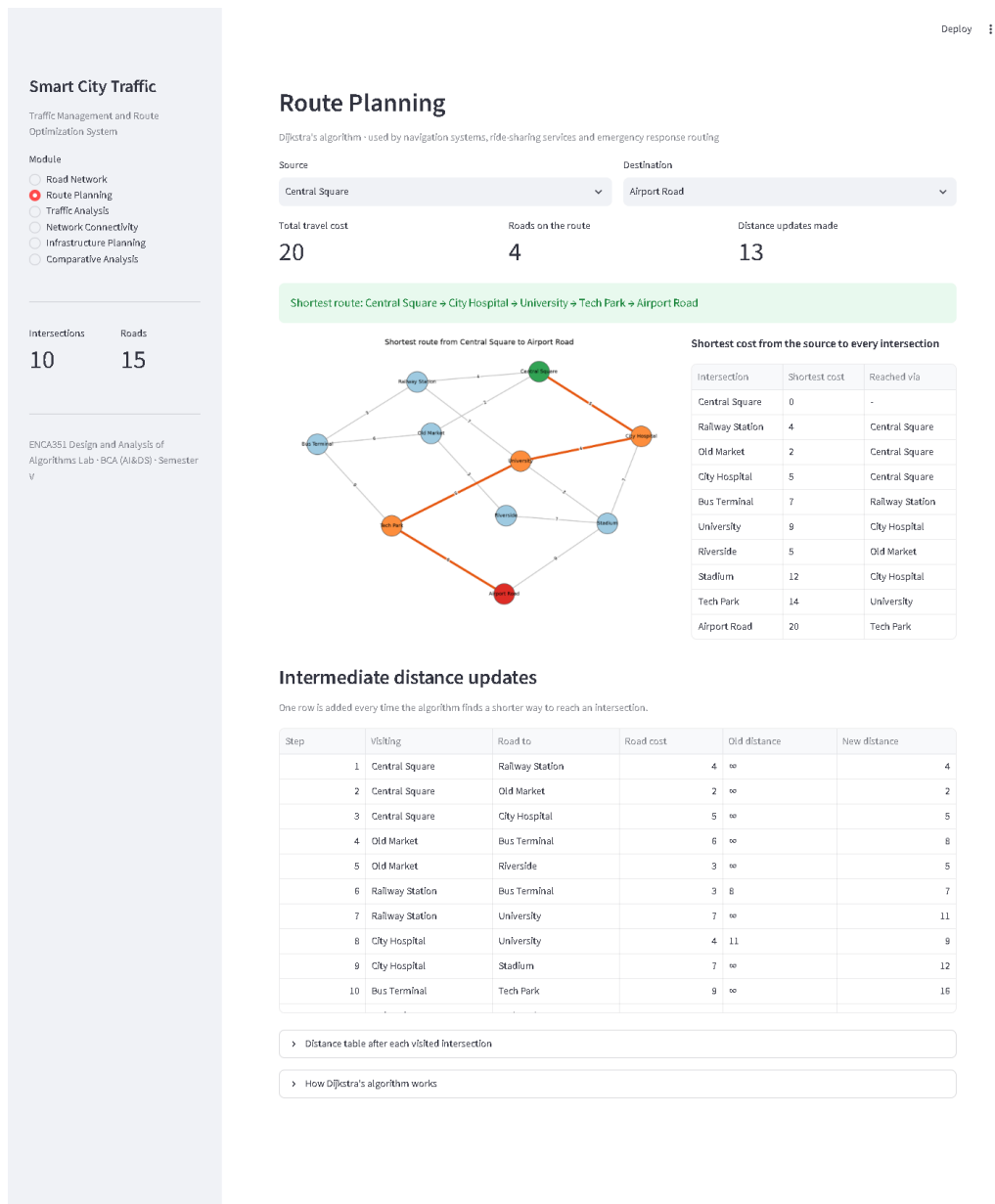


Figure 4: Route Planning page with the intermediate distance updates

3.3 Traffic analysis with the Bellman-Ford algorithm

On the unchanged city the Bellman-Ford algorithm produces the same route and cost as Dijkstra (Central Square → City Hospital → University → Tech Park → Airport Road, 20 minutes). It ran 2 passes out of a possible 9: the second pass changed nothing, so the algorithm stopped there. Table 9 shows the distance table after each pass.

Table 9: Bellman-Ford distance table after each pass (original city)

Pass	Central Sq.	Railway St.	Old Market	Hospital	Bus Term.	University	Riverside	Stadium	Tech Park	Airport Rd.
1	0	4	2	5	7	9	5	12	14	20
2	0	4	2	5	7	9	5	12	14	20

Scenario 1: peak-hour congestion. Every road on the current shortest route becomes three times slower, which is what happens when everybody uses the same main route at the same time.

Table 10: Roads slowed down in the peak-hour scenario

Road	Original cost	Congested cost
------	---------------	----------------

Central Square – City Hospital	5	15
City Hospital – University	4	12
University – Tech Park	5	15
Tech Park – Airport Road	6	18

Table 11: Route comparison for the peak-hour scenario

	Route	Travel cost (min)
Original	Central Square → City Hospital → University → Tech Park → Airport Road	20
After congestion	Central Square → Old Market → Riverside → Stadium → Airport Road	21
Difference	route changed	+1 (5% more)

The commuter is diverted to a completely different route through Old Market, Riverside and Stadium, and loses only 1 minute, because the city offers a good alternative. Bellman-Ford ran 2 passes on the congested city.

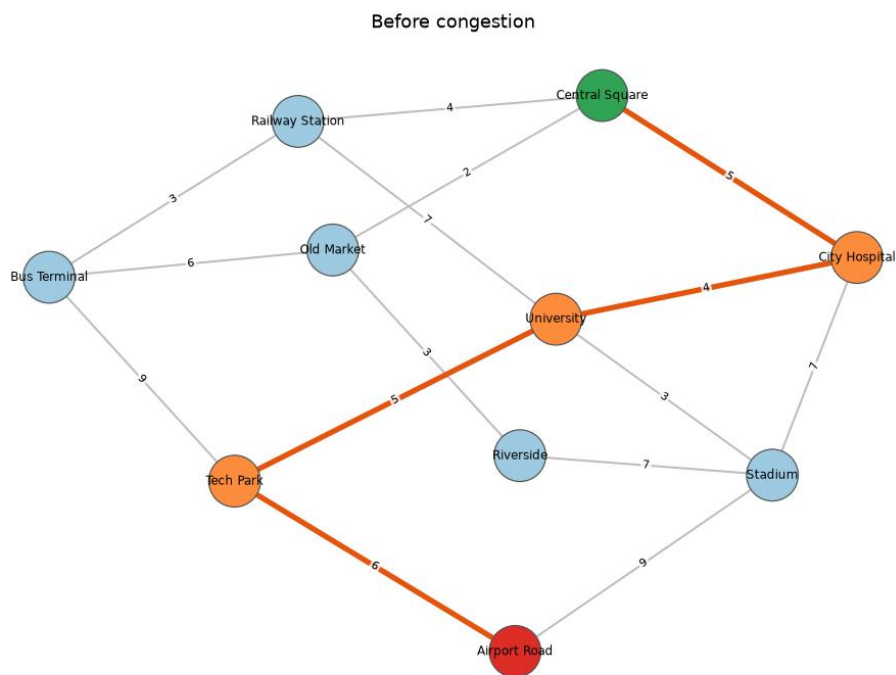


Figure 5: Route before congestion

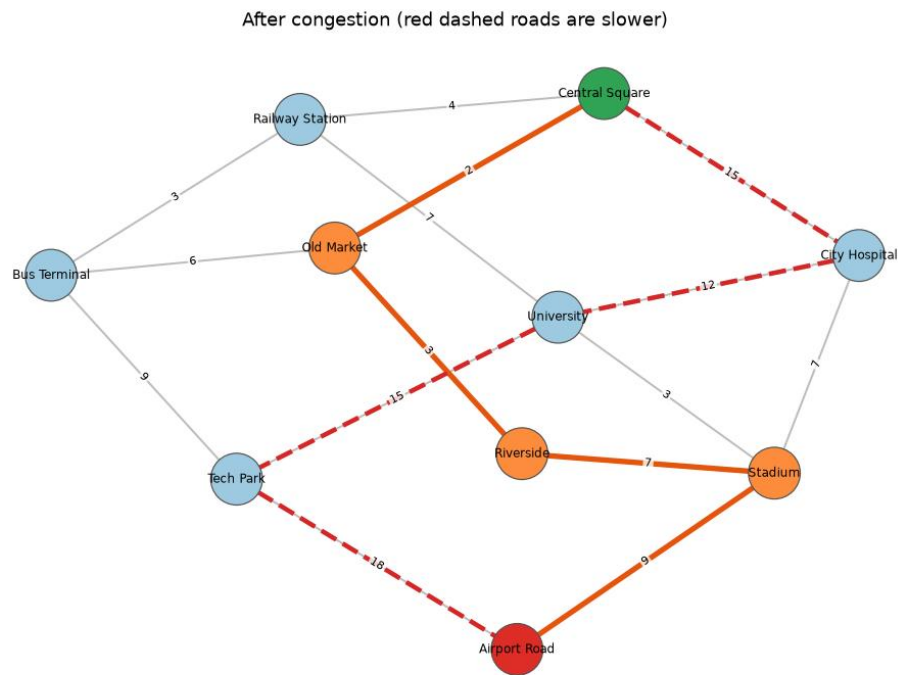


Figure 6: Route after peak-hour congestion; the red dashed roads became slower

Scenario 2: road closure. An accident closes the road between University and Tech Park. The best route becomes Central Square → City Hospital → Stadium → Airport Road at 21 minutes, again only 1 minute more than before. Closing a road removes it from the graph, so Bellman-Ford is simply run on a city with one road fewer.

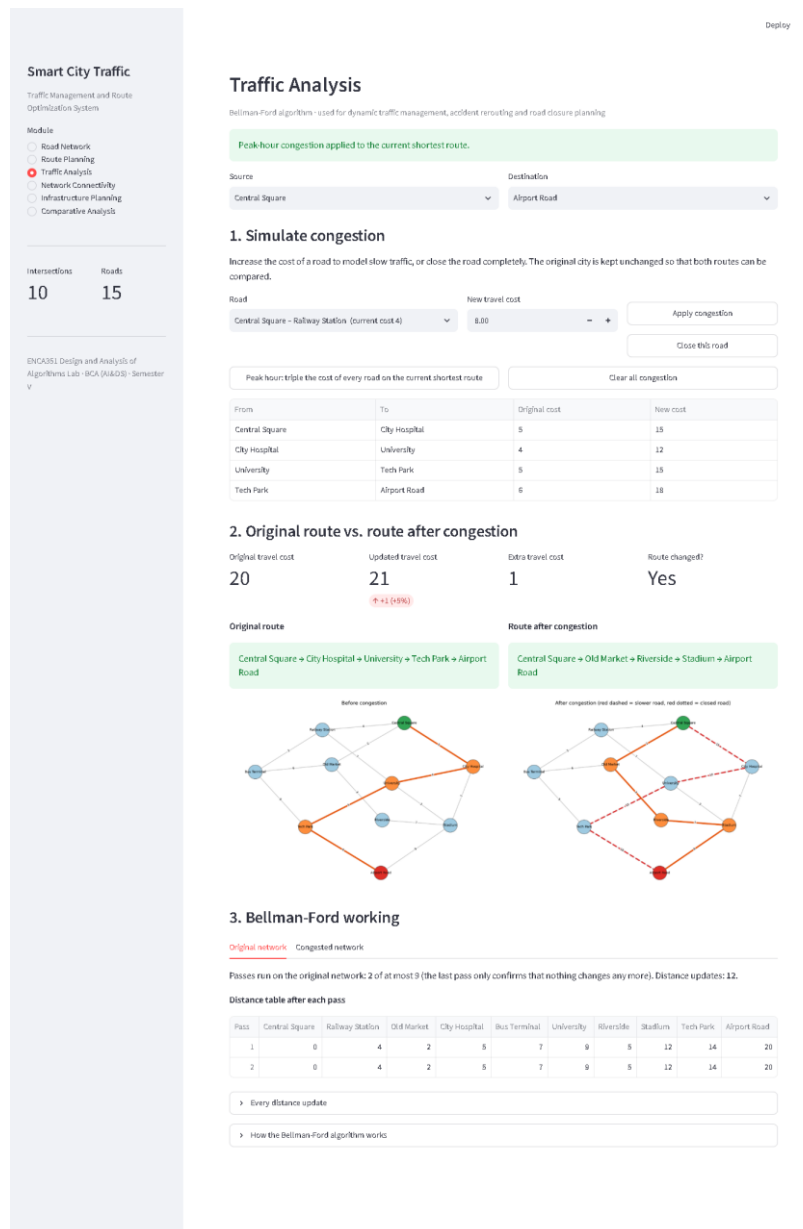


Figure 7: Traffic Analysis page comparing the original and congested routes

3.4 Network connectivity with BFS and DFS

Both traversals start at Central Square. BFS visits the intersections level by level: Central Square → Railway Station → Old Market → City Hospital → Bus Terminal → University → Riverside → Stadium → Tech Park → Airport Road. DFS follows one road as far as it goes before backtracking: Central Square → Railway Station → Bus Terminal → Old Market → Riverside → Stadium → City Hospital → University → Tech Park → Airport Road. Both reach all 10 intersections, so the sample city is connected. Tables 12 and 13 show the queue and the recursion route at every step.

Table 12: Breadth first search step by step

Step	Visiting	Level	Newly discovered	Queue after this step
1	Central Square	0	Railway Station, Old Market, City Hospital	Railway Station, Old Market, City Hospital
2	Railway Station	1	Bus Terminal, University	Old Market, City Hospital, Bus Terminal, University
3	Old Market	1	Riverside	City Hospital, Bus Terminal, University, Riverside
4	City Hospital	1	Stadium	Bus Terminal, University, Riverside, Stadium
5	Bus Terminal	2	Tech Park	University, Riverside, Stadium, Tech Park
6	University	2	-	Riverside, Stadium, Tech Park

7	Riverside	2	-	Stadium, Tech Park
8	Stadium	2	Airport Road	Tech Park, Airport Road
9	Tech Park	3	-	Airport Road
10	Airport Road	3	-	(empty)

Table 13: Depth first search step by step

Step	Visiting	Depth	Route from the start
1	Central Square	0	Central Square
2	Railway Station	1	Central Square -> Railway Station
3	Bus Terminal	2	Central Square -> Railway Station -> Bus Terminal
4	Old Market	3	Central Square -> Railway Station -> Bus Terminal -> Old Market
5	Riverside	4	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside
6	Stadium	5	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside -> Stadium
7	City Hospital	6	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside -> Stadium -> City Hospital
8	University	7	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside -> Stadium -> City Hospital -> University
9	Tech Park	8	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside -> Stadium -> City Hospital -> University -> Tech Park
10	Airport Road	9	Central Square -> Railway Station -> Bus Terminal -> Old Market -> Riverside -> Stadium -> City Hospital -> University -> Tech Park -> Airport Road

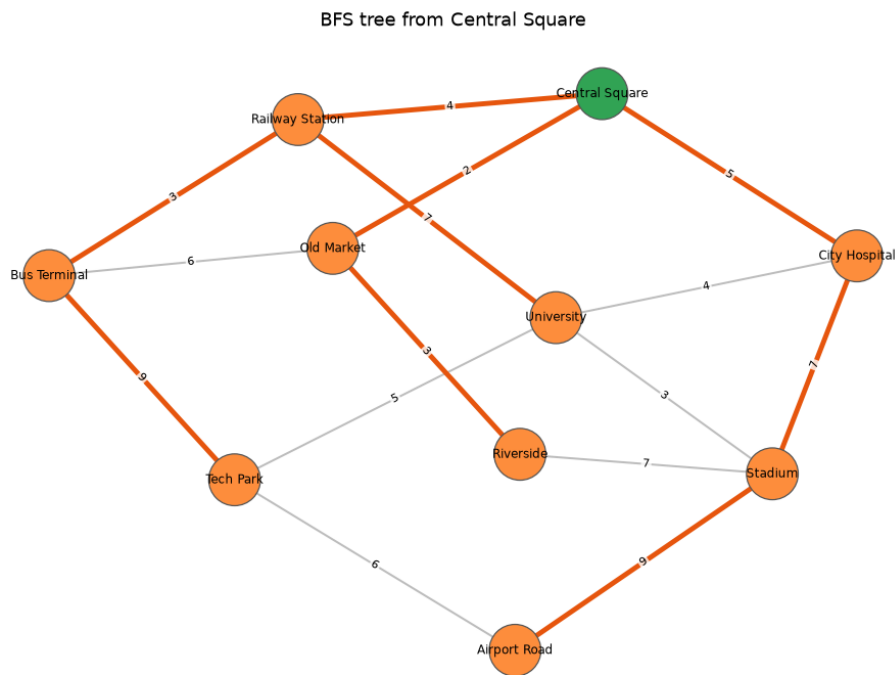


Figure 8: BFS tree from Central Square (orange roads show how each intersection was first reached)

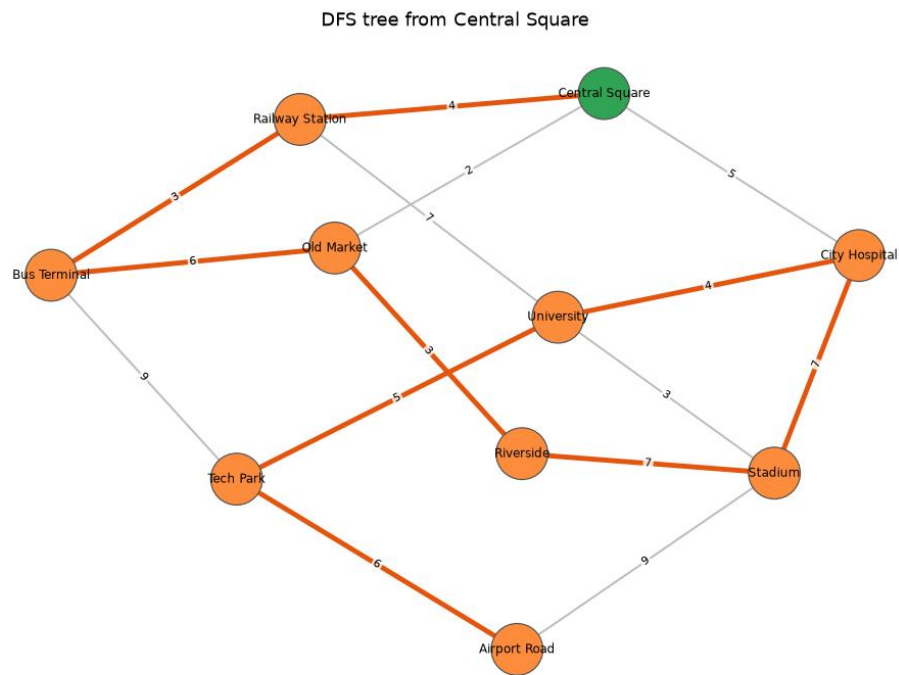


Figure 9: DFS tree from Central Square

Connectivity also reveals how a city breaks apart. When both roads into Airport Road are closed, BFS from Central Square reaches only 9 intersections and the city splits into 2 regions: one with 9 intersections and one containing only Airport Road. The application reports such unreachable intersections and lists the regions.

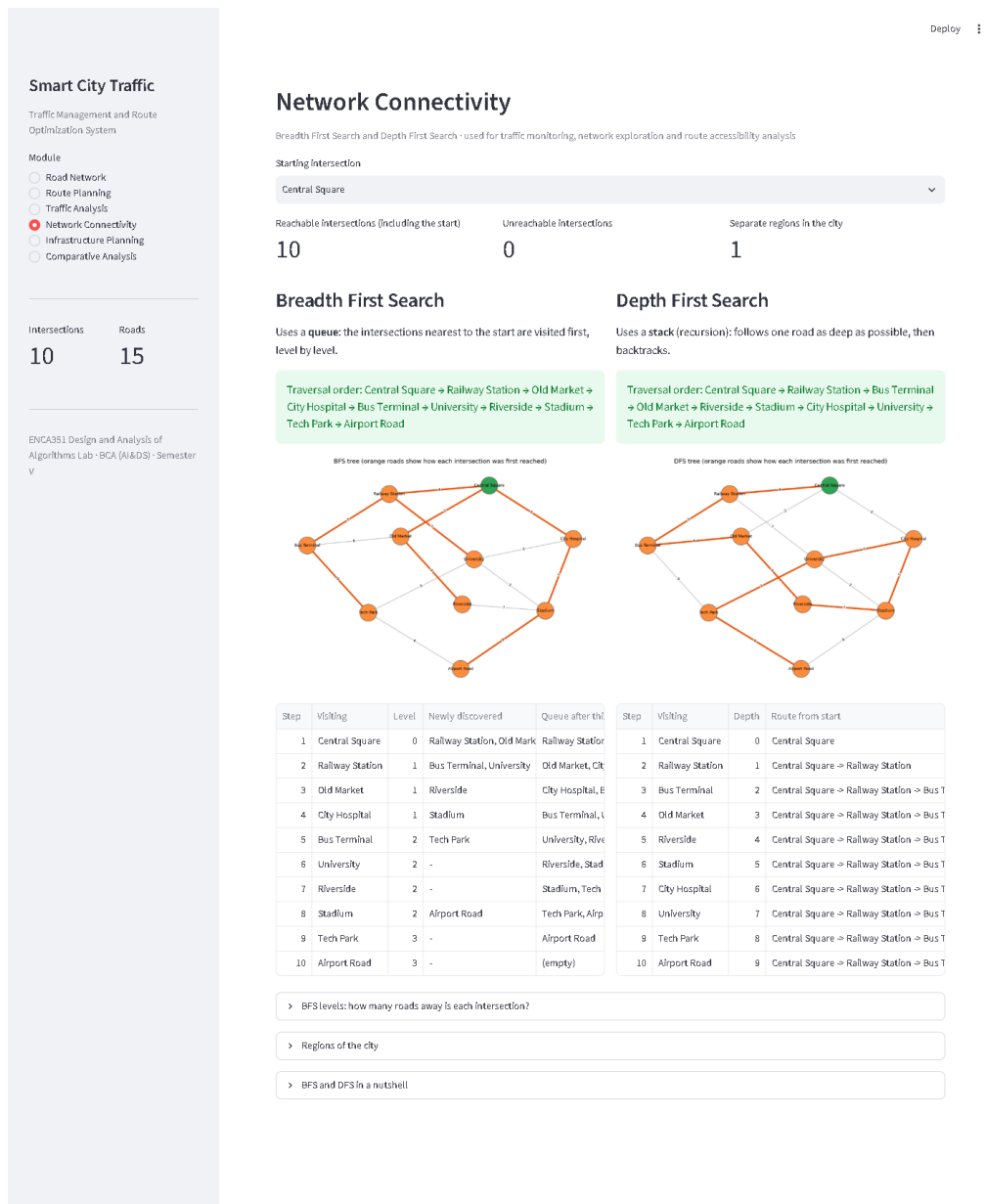


Figure 10: Network Connectivity page

3.5 Infrastructure planning with Prim's and Kruskal's algorithms

Building all 15 roads of the sample city would cost 80 units. Both Prim's and Kruskal's algorithms select 9 roads with a total cost of 35, a saving of 45, and in this city they choose exactly the same roads (yes). Table 14 lists the selected roads and Tables 15 and 16 show every decision of the two algorithms.

Table 14: Roads selected for the minimum spanning tree

From	To	Cost
Central Square	Old Market	2
Railway Station	Bus Terminal	3
Old Market	Riverside	3
University	Stadium	3
Central Square	Railway Station	4
City Hospital	University	4
Central Square	City Hospital	5
University	Tech Park	5
Tech Park	Airport Road	6

Table 15: Prim's algorithm step by step (starting at Central Square)

Step	Road	Cost	Decision
1	Central Square - Old Market	2	Added to the tree
2	Old Market - Riverside	3	Added to the tree
3	Central Square - Railway Station	4	Added to the tree
4	Railway Station - Bus Terminal	3	Added to the tree
5	Central Square - City Hospital	5	Added to the tree
6	City Hospital - University	4	Added to the tree
7	University - Stadium	3	Added to the tree
8	University - Tech Park	5	Added to the tree
9	Old Market - Bus Terminal	6	Skipped (both ends already connected)
10	Tech Park - Airport Road	6	Added to the tree

Table 16: Kruskal's algorithm step by step (roads in order of cost)

Step	Road	Cost	Decision
1	Central Square - Old Market	2	Added to the tree
2	Railway Station - Bus Terminal	3	Added to the tree
3	Old Market - Riverside	3	Added to the tree
4	University - Stadium	3	Added to the tree
5	Central Square - Railway Station	4	Added to the tree
6	City Hospital - University	4	Added to the tree
7	Central Square - City Hospital	5	Added to the tree
8	University - Tech Park	5	Added to the tree
9	Old Market - Bus Terminal	6	Rejected (would create a cycle)
10	Tech Park - Airport Road	6	Added to the tree

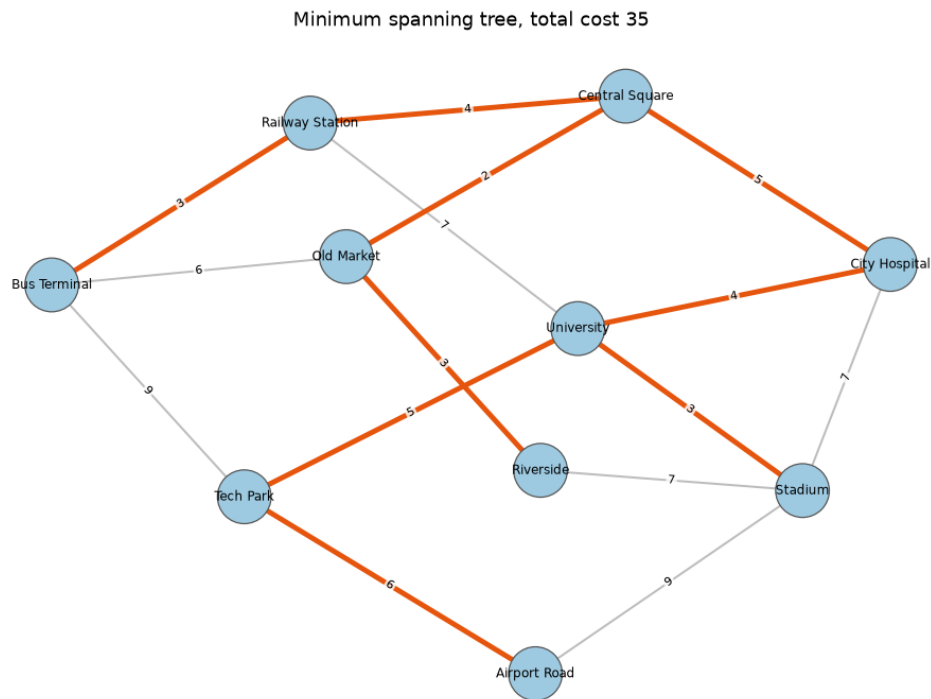


Figure 11: Minimum spanning tree of the sample city (orange roads)

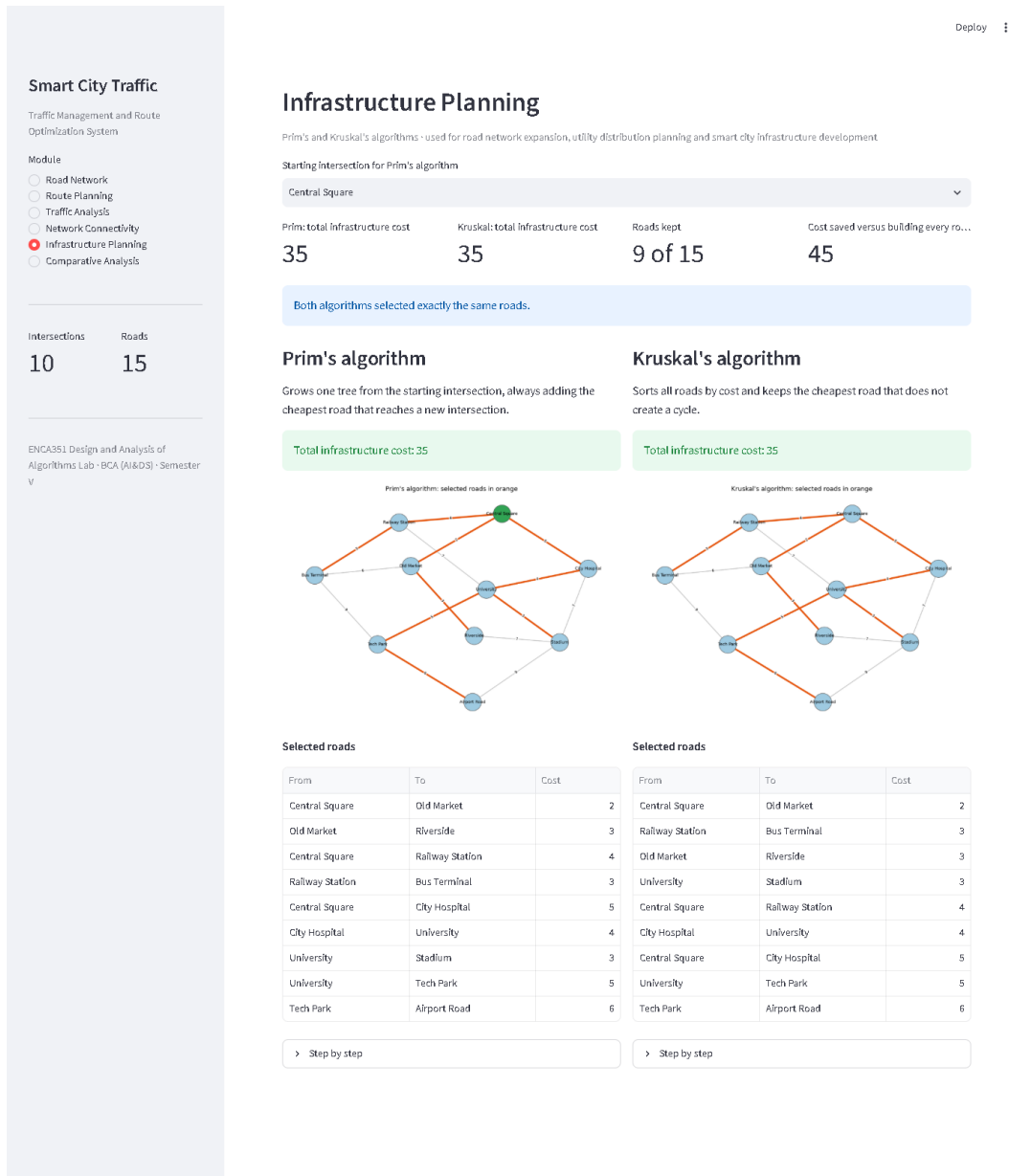


Figure 12: Infrastructure Planning page

3.6 Measurements on networks of different sizes

Random but connected cities with 10, 25, 50 and 100 intersections were generated with about two roads per intersection and costs between 1 and 20. Every algorithm was run on every city. The execution time is the median of 30 runs with the step recording switched off; the peak memory is measured with Python's tracemalloc module. The route is always from the first to the last intersection of the city. Table 17 contains every measurement; Figures 15 and 16 plot them.

Table 17: All measurements of the comparative analysis

Intersections	Roads	Algorithm	Time (ms)	Peak memory (KB)	Result
10	20	Dijkstra	0.0120	1.5	route cost 27 using 2 roads
10	20	Bellman-Ford	0.0272	3.4	route cost 27 using 2 roads
10	20	BFS	0.0068	2.2	10 intersections reached
10	20	DFS	0.0083	2.7	10 intersections reached
10	20	Prim	0.0147	1.4	9 roads, total cost 43
10	20	Kruskal	0.0236	3.8	9 roads, total cost 43
25	50	Dijkstra	0.0321	4.5	route cost 18 using 5 roads
25	50	Bellman-Ford	0.0943	4.5	route cost 18 using 5 roads

25	50	BFS	0.0173	5.1	25 intersections reached
25	50	DFS	0.0186	5.9	25 intersections reached
25	50	Prim	0.0403	3.4	24 roads, total cost 143
25	50	Kruskal	0.0625	4.6	24 roads, total cost 143
50	100	Dijkstra	0.0720	6.2	route cost 19 using 7 roads
50	100	Bellman-Ford	0.1918	13.9	route cost 19 using 7 roads
50	100	BFS	0.0335	7	50 intersections reached
50	100	DFS	0.0368	9.7	50 intersections reached
50	100	Prim	0.0812	3.7	49 roads, total cost 262
50	100	Kruskal	0.1258	13.4	49 roads, total cost 262
100	200	Dijkstra	0.1595	18	route cost 31 using 3 roads
100	200	Bellman-Ford	0.4460	17.3	route cost 31 using 3 roads
100	200	BFS	0.0702	17.8	100 intersections reached
100	200	DFS	0.0744	21.7	100 intersections reached
100	200	Prim	0.1648	12.7	99 roads, total cost 663
100	200	Kruskal	0.2651	15.5	99 roads, total cost 663

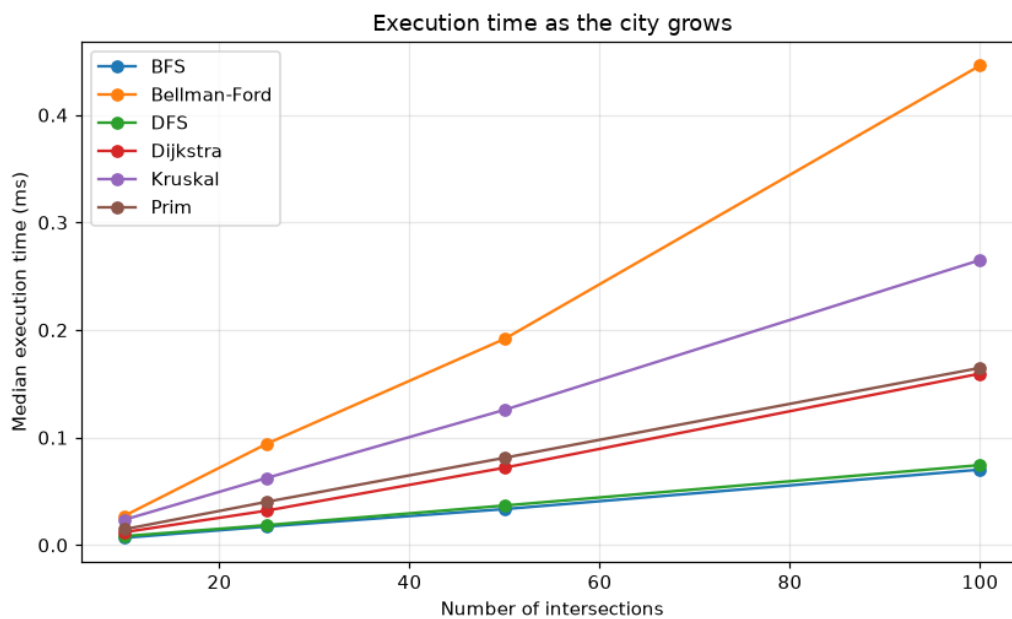


Figure 13: Execution time as the city grows

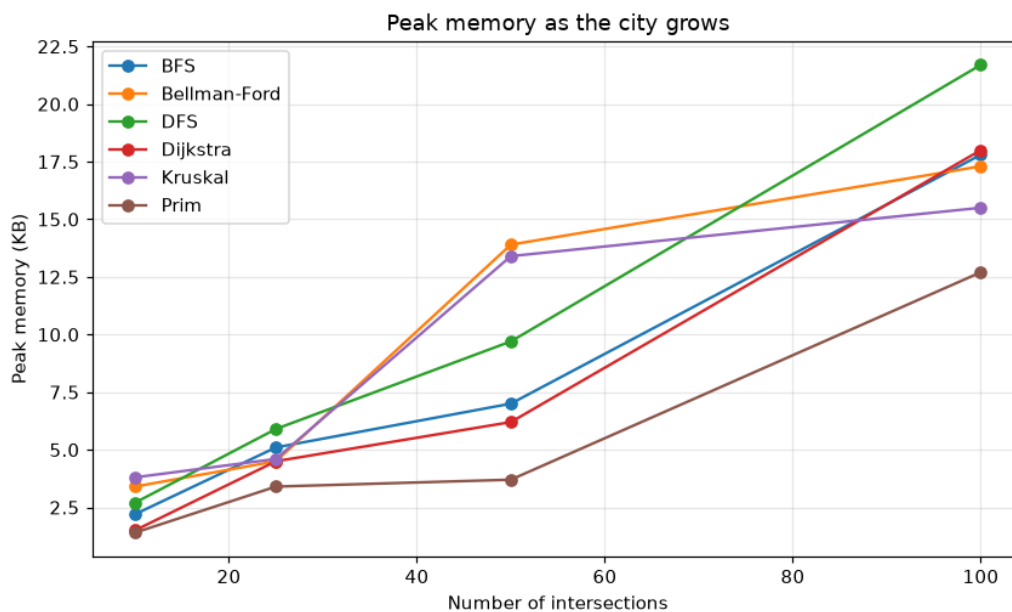


Figure 14: Peak memory as the city grows

Smart City Traffic

Traffic Management and Route Optimization System

Module

- ☐ Road Network
- ☐ Route Planning
- ☐ Traffic Analysis
- ☐ Network Connectivity
- ☐ Infrastructure Planning
- ☒ Comparative Analysis

Intersections

10

Roads

15

ENCA311 Design and Analysis of Algorithms Lab - BCA (NAAG) - Semester V

Deploy

Comparative Analysis

Random but connected cities of different sizes are generated, and every algorithm is run on each of them. The same functions are used in `project_notebooks/compare.py`.

Network sizes (number of intersections)

Repeats per timing

5

Run experiment

All measurements

Execution time is the median of the repeated runs with the step recording switched off. The CSV download contains every recorded column.

Intersections	Roads	Category	Algorithm	Execution time (ms)	Peak memory (KB)	Result
10	20	Shortest path	Dijkstra	0.0226	1.7	route cost 27 using 2 roads
10	20	Shortest path	Bellman-Ford	0.0269	3.6	route cost 27 using 2 roads
10	20	Traversal	BFS	0.0078	2.4	10 intersections reached
10	20	Traversal	DFS	0.0096	2.7	10 intersections reached
10	20	Minimum spanning tree	Prim	0.0289	1.5	9 roads, total cost 43
10	20	Minimum spanning tree	Kruskal	0.0294	3.9	9 roads, total cost 43
25	50	Shortest path	Dijkstra	0.0686	4.7	route cost 18 using 5 roads
25	50	Shortest path	Bellman-Ford	0.1061	4.7	route cost 18 using 5 roads
25	50	Traversal	BFS	0.0317	5.2	25 intersections reached
25	50	Traversal	DFS	0.0185	6	25 intersections reached

Download results as CSV

Execution time as the city grows

Number of intersections	BFS	Bellman-Ford	DFS	Dijkstra	Kruskal	Prim
20	0.0078	0.0269	0.0096	0.0226	0.0289	0.0294
50	0.0317	0.1061	0.0185	0.0686	0.1061	0.1061
100	0.0793	0.178	0.0815	0.178	0.178	0.178

Head-to-head comparisons

BFS vs DFSDijkstra vs Bellman-FordPrim vs Kruskal

Intersections	Roads	BFS time (ms)	BFS memory (KB)	DFS time (ms)	DFS memory (KB)	Faster
10	20	0.0078	2.4	0.0096	2.7	BFS
25	50	0.0317	5.2	0.0185	6	DFS
50	100	0.0435	7.1	0.0383	9.8	DFS
100	200	0.0793	17.8	0.0815	21.7	BFS

Both traversals visit every reachable intersection once, so their running time grows the same way ($O(V + E)$) and the measured times are almost identical. When one of them looks faster it is timing noise. BFS is the better fit for traffic monitoring because it also gives the number of roads to every intersection, which shows how far a disruption spreads. DFS suits region-by-region exploration and cycle detection, but its recursion depth grows with the longest route.

Figure 15: Comparative Analysis page of the application

Smart City Traffic Management and Route Optimization System · Page 20

4. Analysis

4.1 Comparison of graph algorithms

The three head-to-head comparisons below come from the measurements in Table 17. All timings are fractions of a millisecond, so the ratios between algorithms matter more than the absolute values, which would change on a different computer.

Table 18: BFS versus DFS

Intersections	Roads	BFS time (ms)	DFS time (ms)	BFS memory (KB)	DFS memory (KB)	Faster
10	20	0.0068	0.0083	2.2	2.7	BFS
25	50	0.0173	0.0186	5.1	5.9	BFS
50	100	0.0335	0.0368	7	9.7	BFS
100	200	0.0702	0.0744	17.8	21.7	BFS

BFS versus DFS. Both traversals visit every reachable intersection exactly once and look at every road twice, so both run in $O(V + E)$ time, and the measurements agree: the two are within a few microseconds of each other at every size, and the memory used grows the same way. For traffic management BFS is the more useful of the two because it also produces the number of roads from the start to every intersection, which tells a control room how far a disruption spreads. DFS is a natural fit for exploring the network region by region and for detecting cycles, but the recursive version goes one call deeper for every intersection on the current route, which becomes a limitation on very long chains of roads.

Table 19: Dijkstra versus Bellman-Ford

Intersections	Roads	Dijkstra time (ms)	Bellman-Ford time (ms)	Bellman-Ford / Dijkstra	Bellman-Ford passes	Dijkstra memory (KB)	Bellman-Ford memory (KB)
10	20	0.0120	0.0272	2.3×	2	1.5	3.4
25	50	0.0321	0.0943	2.9×	4	4.5	4.5
50	100	0.0720	0.1918	2.7×	4	6.2	13.9
100	200	0.1595	0.4460	2.8×	5	18	17.3

Dijkstra versus Bellman-Ford. Both algorithms found the same route and the same cost on every city, as they must. Dijkstra was the faster one at every size, between 2.3 and 2.9 times faster than Bellman-Ford. The reason is the amount of work per road: Dijkstra relaxes each road once, when the intersection it leaves is settled, so its cost is $O((V + E) \log V)$. Bellman-Ford relaxes every road in every pass, so its cost is $O(V \times E)$ in the worst case. The early stop kept the number of passes small, between 2 and 5 instead of $V - 1$, which is why the gap did not grow even wider on these random cities. For live navigation Dijkstra is clearly the right choice. Bellman-Ford still earns its place in the traffic analysis module: it is simple to rerun after a planner edits road costs and its pass-by-pass table is easy to explain and to check by hand. It would also tolerate negative costs and detect negative cycles, which Dijkstra cannot do; in this project every road cost is positive, so that ability is a safety net rather than a daily need.

Table 20: Prim versus Kruskal

Intersections	Roads	Prim time (ms)	Kruskal time (ms)	Kruskal / Prim	Prim memory (KB)	Kruskal memory (KB)
10	20	0.0147	0.0236	1.6×	1.4	3.8
25	50	0.0403	0.0625	1.6×	3.4	4.6
50	100	0.0812	0.1258	1.5×	3.7	13.4
100	200	0.1648	0.2651	1.6×	12.7	15.5

Prim versus Kruskal. Both algorithms reached the same minimum total cost on every city, as they must. Prim was faster at every size, by a factor of 1.5 to 1.6, and usually used less memory. Prim only pushes

the roads that leave the growing tree onto its heap, while Kruskal must first sort every road and then run a union-find check on each one. Kruskal's advantage is clarity rather than speed: its sorted list of roads with an accept or reject decision for each one (Table 16) is the easier result to explain to city planners, and it works directly on a plain list of roads without needing an adjacency structure.

4.2 Traffic optimization observations

- A route that is optimal at one moment is not optimal for long. Tripling the cost of the roads on the best route pushed the commuter onto a completely different route through the other side of the city, and closing a single road did the same. A traffic system therefore has to recompute routes whenever costs change, which is exactly what the Bellman-Ford module does on a copy of the city.
- Alternative routes limit the damage. In both scenarios the extra travel cost was only one minute because the sample city has parallel routes. A city with a single main road would see far larger increases, which argues for keeping alternatives even if they are not used in normal traffic.
- Showing the working builds trust. The intermediate distance updates and the pass-by-pass tables make it possible to check a route by hand, which matters when an administrator has to justify a diversion.
- Rerouting must be cheap. Even Bellman-Ford took well under a millisecond on 100 intersections, so a city could recompute routes for every reported change in real time; for larger networks Dijkstra's better scaling would matter.

4.3 Infrastructure planning insights

- The minimum spanning tree of the sample city keeps 9 of 15 roads and costs 35 instead of 80, a saving of 56 percent, while still connecting every intersection.
- The spanning tree has no redundancy: every road in it is a bridge, so a single closure splits the city. The connectivity module showed how quickly a city breaks into regions when roads are removed. A real plan would start from the tree and then add the cheapest roads that create alternative routes for the busiest connections.
- Prim and Kruskal always agree on the total cost, so either can be used; the choice depends on whether a planner prefers to grow the network from a chosen point (Prim) or to review roads in order of cost (Kruskal).

4.4 Suitability of each algorithm for traffic management

Table 21: Suitability of each algorithm

Algorithm	Best suited for	Why
Dijkstra	Live route planning for commuters and emergency vehicles	Fastest and scales best; needs positive costs, which travel times always are
Bellman-Ford	Recomputing routes after administrators edit road costs	Simple to rerun on an edited graph; its pass-by-pass working is easy to explain and check
BFS	Monitoring how far a disruption spreads	Gives the number of roads from the start to every intersection
DFS	Exploring a region or checking whether a route exists	Same cost as BFS; naturally follows one road to its end
Prim	Growing a new network from an existing hub	Fastest MST algorithm in the experiments; expands from a chosen intersection
Kruskal	Reviewing candidate roads in order of construction cost	Produces a sorted accept or reject list that is easy to explain

5. Conclusion

5.1 Key findings

- A city road network fits naturally into a weighted graph. The adjacency list keeps every operation simple and the adjacency matrix gives a readable overview.
- Dijkstra's algorithm found the fastest route in the sample city (20 minutes from Central Square to Airport Road) and its intermediate distance updates make the result easy to verify.
- The Bellman-Ford module recomputed routes after congestion and closures and quantified the extra travel cost; in both scenarios a good alternative route limited the loss to one minute.
- BFS and DFS showed which intersections are reachable, in what order, and how the city splits into regions when roads close.
- Prim's and Kruskal's algorithms connected every intersection for 35 instead of 80, and always agreed with each other.
- In the experiments Dijkstra was two to three times faster than Bellman-Ford, Prim was faster than Kruskal, and BFS and DFS were indistinguishable. Every algorithm finished in under half a millisecond on a city of 100 intersections.

5.2 Benefits of graph algorithms in transportation systems

Graph algorithms turn a map into something a computer can reason about. They give exact answers, not estimates: the route Dijkstra returns is provably the cheapest one for the given costs, and the spanning tree Kruskal builds is provably the cheapest way to connect the city. They are fast enough to run continuously as conditions change, they show their working so that decisions can be checked, and the same code serves commuters, control rooms and planners. Because the algorithms only see vertices, edges and weights, the same system works for a metro network, a delivery fleet or a set of utility lines.

5.3 Future improvements

- One-way roads, which need a directed graph and small changes to the traversal and MST modules.
- Live costs from traffic sensors or a map service, so that congestion is measured instead of simulated.
- A* search with a geographic heuristic to speed up route planning on city-scale networks.
- Traffic signal timing at intersections, modelled as a waiting cost on the vertices.
- Resilience planning: start from the minimum spanning tree and add the cheapest roads that give every busy connection a second route.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th edition, MIT Press, 2022. Chapters on elementary graph algorithms, minimum spanning trees and single-source shortest paths.
- [2] J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2006. Chapters 3 and 4.
- [3] Python Software Foundation, Python 3 documentation, <https://docs.python.org/3/>
- [4] Streamlit Inc., Streamlit documentation, <https://docs.streamlit.io/>
- [5] NetworkX developers, NetworkX documentation, <https://networkx.org/documentation/stable/>
- [6] Matplotlib developers, Matplotlib documentation, <https://matplotlib.org/stable/>

Appendix A: Repository contents

Table 22: Files in the GitHub repository

File or folder	Contents
README.md	Overview, setup and usage instructions, results summary
app.py	Streamlit web application
road_network.py	RoadNetwork graph class, adjacency list and matrix, CSV load and save
route_planner.py	Dijkstra's algorithm
traffic_analysis.py	Bellman-Ford algorithm and congestion simulation
connectivity_module.py	Breadth first search, depth first search, connected regions
mst_planner.py	Prim's and Kruskal's algorithms
experiments.py	Comparative analysis helpers
visualizer.py	Map drawing and table formatting
project_notebook.ipynb	Walk-through of every module and the comparative analysis, saved with outputs
results/	experiment_results.csv and the figures produced by the notebook
screenshots/	Screenshots of the running application
report/	This report
requirements.txt	streamlit, networkx, matplotlib, pandas, numpy, jupyter

Appendix B: Running the project

1. Create and activate a virtual environment: `python -m venv .venv`, then `.venv\Scripts\activate`.
2. Install the packages: `pip install -r requirements.txt`.
3. Start the application: `streamlit run app.py`, then open `http://localhost:8501`.
4. Open the notebook: `jupyter notebook project_notebook.ipynb`.